

INTERMEDIATE - CODE GENERATION

Variants of Syntax trees

Nodes in a syntax tree represents constructs in the source program, the children of a node represent the meaningful components of a construct. A Directed Acyclic Graph (DAG) for an expression identifier the common subexpression of the expressions.

Directed Acyclic Graphs for Expressions

Like the syntax tree for an expression, a DAG has leaves corresponding to atomic operands and interior nodes corresponding to operators. A DAG not only represents expressions succinctly, it gives the compiler important clues regarding the generation of efficient code to evaluate the expressions.

$$a + a * (b - c) + (b - c) * d$$

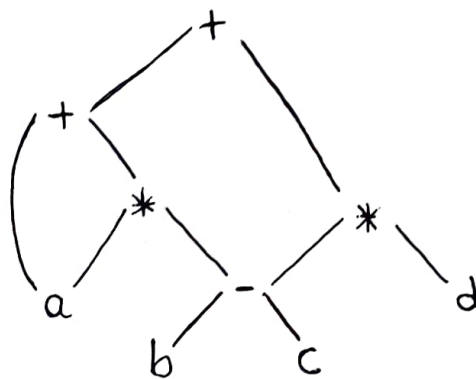


Fig:- 1

DAG for the expression $a + a * (b - c) + (b - c) * d$

Production	Semantic Rules
$E \rightarrow E_1 + T$	$E.\text{node} = \text{new Node}('+', E_1.\text{node}, T.\text{node})$
$E \rightarrow E_1 - T$	$E.\text{node} = \text{new Node}('-', E_1.\text{node}, T.\text{node})$
$E \rightarrow T$	$E.\text{node} = T.\text{node}$
$T \rightarrow (E)$	$T.\text{node} = E.\text{node}$
$T \rightarrow \text{id}$	$T.\text{node} = \text{new Leaf}(\text{id}, \text{id}.\text{entry})$
$T \rightarrow \text{num}$	$T.\text{node} = \text{new Leaf}(\text{num}, \text{num}.\text{val})$

Syntax directed definition to produce Syntax tree or DAG

- 1) $P_1 = \text{Leaf}(\text{id}, \text{entry } b)$
- 2) $P_2 = \text{Leaf}(\text{id}, \text{entry } c)$
- 3) $P_3 = \text{Node}('-', P_1, P_2)$
- 4) $P_4 = \text{Leaf}(\text{id}, \text{entry } a)$
- 5) $P_5 = \text{Node}('*', P_4, P_3)$
- 6) $P_6 = \text{Leaf}(\text{id}, \text{entry } d)$
- 7) $P_7 = \text{Node}('*', P_6, P_3)$
- 8) $P_8 = \text{Node}('+', P_4, P_5)$
- 9) $P_9 = \text{Node}('+', P_8, P_7)$

Steps for constructing the DAG i.e., Fig:-1

The fig.1 shows the DAG for the expression

$$a + a * (b - c) + (b - c) * d$$

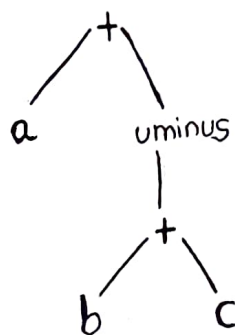
The leaf for a has two parents, because a appears twice in the expression. More interestingly the two occurrences of the common subexpression $b - c$ are represented by one node, the node labeled $-$, that node has two parents, representing its two uses in the subexpression $a * (b - c)$ and $(b - c) * d$. Even though b and c appear twice in the complex expression, their nodes each have one parent, since both uses are in the common subexpression $b - c$.

Advantages of DAG

- 1) More compact representation.
- 2) Gives clues regarding generation of efficient code.

Construct a DAG for the expressions given below and also give the sequence of steps for constructing the same

$$a + -(b + c)$$



Q DAG for the expression $a + -(b + c)$

$p_1 = \text{leaf}(\text{id}, \text{entry } b)$

$p_2 = \text{leaf}(\text{id}, \text{entry } c)$

$p_3 = \text{node}('+', p_1, p_2)$

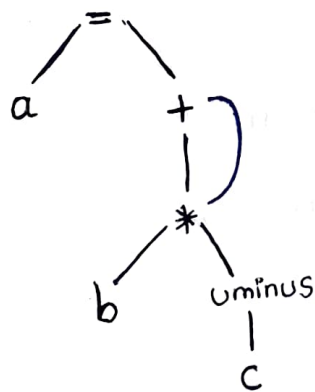
$p_4 = \text{leaf}(\text{id}, \text{entry } \text{uminus } p_3)$

$p_5 = \text{leaf}(\text{id}, \text{entry } a)$

$p_6 = \text{node}('+', p_5, p_4)$

a) Steps for constructing the DAG.

$a = b * -c + b * -c$



b) DAG for the expression $a = b * -c + b * -c$

$p_1 = \text{leaf}(\text{id}, \text{entry } b)$

$p_2 = \text{leaf}(\text{id}, \text{entry } \text{uminus } c)$

$p_3 = \text{node}('*', p_1, p_2)$

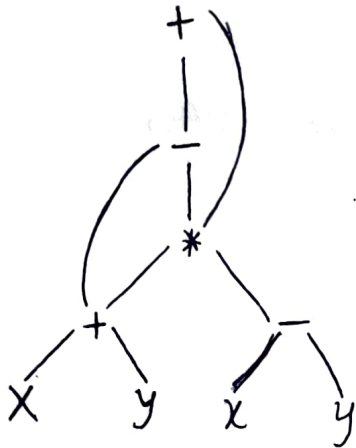
$p_4 = \text{node}('+', p_3, p_3)$

$p_5 = \text{leaf}(\text{id}, \text{entry } a)$

$p_6 = \text{node}('=', p_5, p_4)$

c) Steps for constructing the DAG

$$((x+y) - ((x+y) * (x-y))) + ((x+y) * (x-y))$$



c] DAG for the expression $((x+y) - ((x+y) * (x-y))) + ((x+y) * (x-y))$

$p_1 = \text{leaf}(\text{id}, \text{entry } x)$

$p_2 = \text{leaf}(\text{id}, \text{entry } y)$

$p_3 = \text{node}('+', p_1, p_2)$

$p_4 = \text{node}('-', p_1, p_2)$

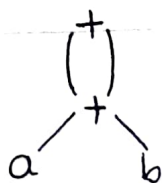
$p_5 = \text{node}('*', p_3, p_4)$

$p_6 = \text{node}('-', p_3, p_5)$

$p_7 = \text{node}('+', p_6, p_5)$

c] Steps for constructing the DAG.

$$a + b + a + b$$



d] DAG for the expression $a + b + a + b$

$p_1 = \text{leaf}(\text{id}, \text{entry } a)$

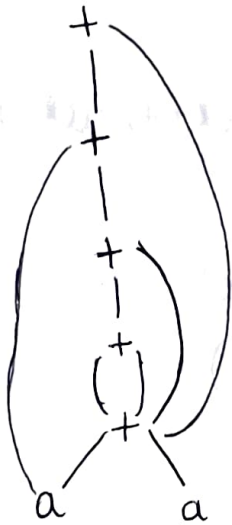
$p_2 = \text{leaf}(\text{id}, \text{entry } b)$

$p_3 = \text{node}('+', p_1, p_2)$

$p_4 = \text{node}('+', p_3, p_3)$

d] Steps for constructing the DAG

$a + a + ((a + a + a + (a + a + a + a)))$



e] DAG for the expression $a + a + (a + a + a + (a + a + a + a))$

$p_1 = \text{leaf}(\text{id}, \text{entry } a)$

$p_2 = \text{leaf}(\text{id}, \text{entry } a)$

$p_3 = \text{node}('+', p_1, p_2)$

$p_4 = \text{node}('+', p_3, p_3)$

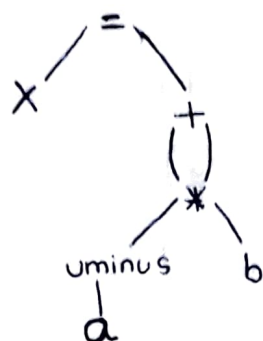
$p_5 = \text{node}('+', p_4, p_3)$

$p_6 = \text{node}('+', p_5, p_1)$

$p_7 = \text{node}('+', p_6, p_3)$

e] Steps for constructing the DAG

$$x = -a * b + -a * b$$



f] DAG for the expression $x = -a * b + -a * b$

$p_1 = \text{leaf}(\text{id}, \text{entry } \text{uminus } a)$

$p_2 = \text{leaf}(\text{id}, \text{entry } b)$

$p_3 = \text{node}('*', p_1, p_2)$

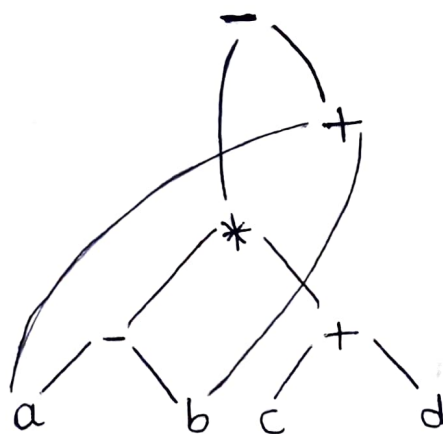
$p_4 = \text{node}('+', p_3, p_3)$

$p_5 = \text{leaf}(\text{id}, \text{entry } x)$

$p_6 = \text{node}('=', p_5, p_4)$

g] steps for constructing the DAG

$$(a-b) * (c+d) - (a+b)$$



g] DAG for the expression $(a-b) * (c+d) - (a+b)$

$P_1 = \text{leaf}(\text{id}, \text{entry } a)$
 $P_2 = \text{leaf}(\text{id}, \text{entry } b)$
 $P_3 = \text{leaf}(\text{id}, \text{entry } c)$
 $P_4 = \text{leaf}(\text{id}, \text{entry } d)$
 $P_5 = \text{node}('-', P_1, P_2)$
 $P_6 = \text{node}('+', P_3, P_4)$
 $P_7 = \text{node}('+', P_1, P_2)$
 $P_8 = \text{node}('*', P_5, P_6)$
 $P_9 = \text{node}('-', P_8, P_7)$

9] Steps for constructing the DAG.

Three - Address Code

In three address code, there is at most one operator on the right side of an instruction. Thus a source-language expression like $x + y * z$ might be translated into the sequence of three address instructions.

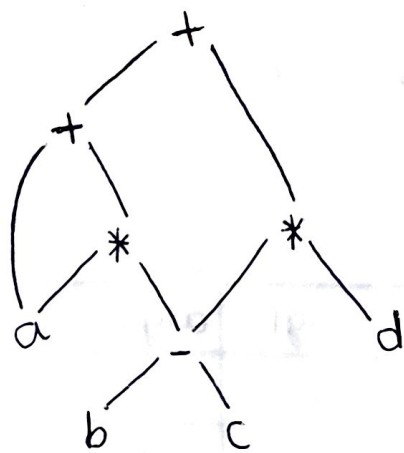
$$t_1 = y * z$$

$$t_2 = x + t_1$$

where t_1 and t_2 are compiler-generated temporary names.

Example :- Three - address code is a linearized representation of a syntax tree or a DAG in which explicit names correspond to the interior nodes of the graph.

$$a + a * (b - c) + (b - c) * d$$



$$t_1 = b - c$$

$$t_2 = a * t_1$$

$$t_3 = a + t_2$$

$$t_4 = t_1 * d$$

$$t_5 = t_3 + t_4$$

a) DAG

b) Three-address code

A DAG and its corresponding three-address code.

Three ways representation of three-address code are:-

1] Quadruples

2] Triples

3] Indirect Triples.

Quadruples:- A quadruple has four fields, which we call OP, arg1, arg2 and result. The OP field contains an internal code for the operator. For instance, the three-address instruction $X = Y + Z$ is represented by placing + in OP, Y in arg1 and Z in arg2, X in result.

$$a = b * -c + b * -c$$

$$t_1 = \text{uminus } C$$

$$t_2 = b * t_1$$

$$t_3 = \text{uminus } C$$

$$t_4 = b * t_3$$

$$t_5 = t_2 + t_4$$

$$a = t_5$$

	OP	arg1	arg2	Result
0	minus	C		t_1
1	*	b	t_1	t_2
2	minus	C		t_3
3	*	b	t_3	t_4
4	+	t_2	t_4	t_5
5	=	t_5		a

a) Three-Address Code

b) Quadruples

Three-Address Code and its Quadruple Representation

Triples:- A Triple has only three fields, which we call OP, arg1 and arg2. Using triples, we refer to the result of an operation $X \text{ OP } Y$ by its position, rather than by an explicit temporary name.

$$a = b * -c + b * -c$$

$$t_1 = \text{minus } c$$

$$t_2 = b * t_1$$

$$t_3 = \text{minus } c$$

$$t_4 = b * t_3$$

$$t_5 = t_2 + t_4$$

$$a = t_5$$

	OP	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

a) Three-Address Code

b) Triples

Three-Address Code and its Triple representation

Indirect Triples:- Indirect Triples consists of a listing of pointers to triples, rather than a listing of triples themselves.

$$a = b * -c + b * -c$$

Instruction

35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)

	OP	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Indirect Triples Representation of three-address code

In the quadruple representation using temporary names the entries in the symbol table against those temporaries can be obtained.

*] The advantage with quadruple representation is that one can quickly access the value of temporary variables using symbol table.

*] The quadruple representation is benefical for code optimization.

*] Indirect Triple saves some amount of space as compared with quadruple representation.

Static Single - Assignment Form

Static Single - assignment form (SSA) is an intermediate representation that facilitates certain code optimization. In SSA all assignments to variables with distinct names, hence the term static single assignment. Below figure * shows the same intermediate program in three - address code and in static single assignment form.

$$p = a + b$$

$$q = p - c$$

$$p = q * d$$

$$p = e - p$$

$$q = p + q$$

$$p_1 = a + b$$

$$q_1 = p_1 - c$$

$$p_2 = q_1 * d$$

$$p_3 = e - p_2$$

$$q_2 = p_3 + q_1$$

a] Three - address code

b] static - single - assignment Form

Fig. * Intermediate Program in three address code and Static Single - Assignment Form

Write the quadruple, Triple and Indirect Triple for the statement.

$$a = b * -c + b * -c$$

$$t_1 = \text{minus } c$$

$$t_2 = b * t_1$$

$$t_3 = \text{minus } c$$

$$t_4 = b * t_3$$

$$t_5 = t_2 + t_4$$

$$a = t_5$$

a) Three-Address Code

	OP	Arg1	Arg2	Result
0	minus	c		t_1
1	*	b	t_1	t_2
2	minus	c		t_3
3	*	b	t_3	t_4
4	+	t_2	t_4	t_5
5	=	t_5		a

b) Quadruple

	OP	Arg1	Arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

b) Triple

Indirect Triples:-

Instruction

35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)

	OP	Arg1	Arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Translate the arithmetic expression
 $a + a * (b - c) + (b - c) * d$ into quadruple, triple and indirect triple.

The three address code will be

$$t_1 = b - c$$

$$t_2 = a * t_1$$

$$t_3 = t_1 * d$$

$$t_4 = t_2 + t_3$$

$$t_5 = a + t_4$$

	OP	Arg1	Arg2	Result
0	-	b	c	t_1
1	*	a	t_1	t_2
2	*	t_1	d	t_3
3	+	t_2	t_3	t_4
4	+	a	t_4	t_5

b] Quadruple

Triple

	OP	ARG1	ARG2
0	-	b	c
1	*	a	(0)
2	*	d	(0)
3	+	(1)	(2)
4	+	a	(3)

Indirect Triple

Instruction

10	(0)
11	(1)
12	(2)
13	(3)
14	(4)

	OP	ARG1	ARG2
0	-	b	c
1	*	a	(0)
2	*	d	(0)
3	+	(1)	(2)
4	+	a	(3)

Write the quadruple, Triple and Indirect triple for the statement

$$X = -a * b + -a * b$$

a] Three address code

$$t_1 = \text{minus } a$$

$$t_2 = t_1 * b$$

$$t_3 = \text{minus } a$$

$$t_4 = t_3 * b$$

$$t_5 = t_2 + t_4$$

$$X = t_5$$

b] Quadruple

	OP	ARG 1	ARG 2	Result
0	minus	a		t ₁
1	*	t ₁	b	t ₂
2	minus	a		t ₃
3	*	t ₃	b	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		X

c] Triple

	OP	ARG 1	ARG 2
0	minus	a	
1	*	(0)	b
2	minus	a	
3	*	(2)	b
4	+	(1)	(3)
5	=	(4)	X

Indirect Triple

11	(0)
12	(1)
13	(2)
14	(3)
15	(4)
16	(5)

	OP	ARG 1	ARG 2
0	minus	a	
1	*	(0)	b
2	minus	a	
3	*	(2)	b
4	+	(1)	(3)
5	=	(4)	X

Control Flow :-

The translation of statement such as if-else statement and while statements is tied to the translation of boolean expression. In programming languages, boolean expression are often used to

a) Alter the flow of control :- Boolean expression are used as conditional expression in statements that alter the flow of control.

For example :- If (E) - then S

The expression E must be true if statement S is reached.

b) Compute logical values :- A boolean expression can represent true or false as values. Such boolean expressions can be evaluated in analogy to arithmetic expression using three address instructions with logical operators.

Flow of Control Statements

In this section we will discuss the translation of Boolean expression into three address code. The control statements are if-then-else and while-do. The grammar for such statements is as shown below.

$$\begin{aligned} S \rightarrow & \text{if } E \text{ then } S_1 \\ & | \text{if } E \text{ then } S_1 \text{ else } S_2 \\ & | \text{while } E \text{ do } S_1 \end{aligned}$$

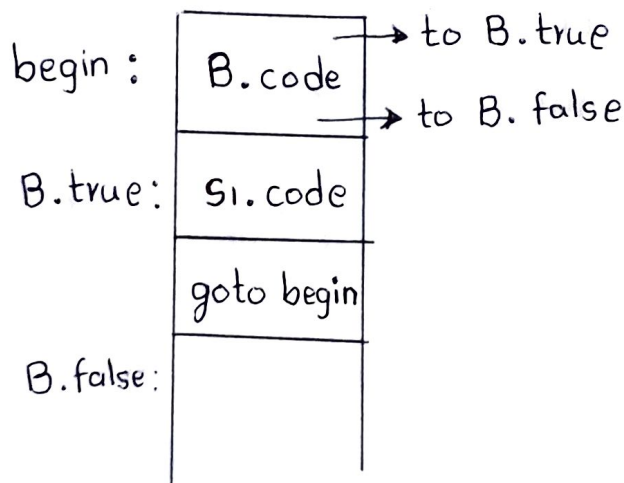
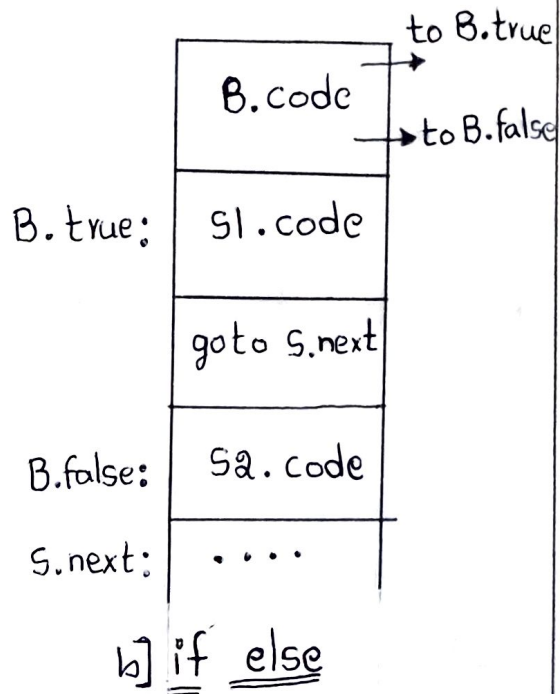
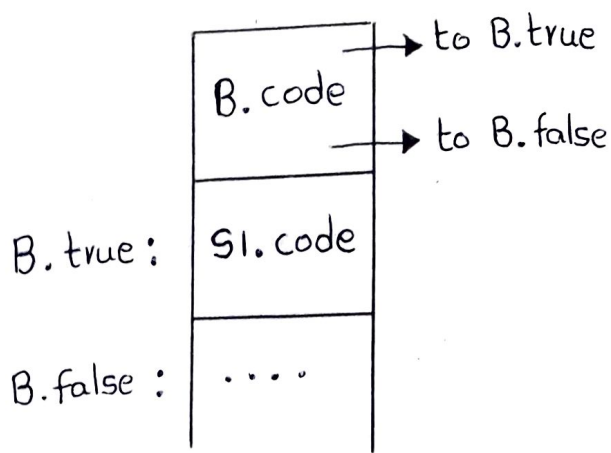


Fig:* Code for if, if-else and while statements.

The translation of if (B) S₁ consists of B.code followed by S₁.code as illustrated in Fig:*(a) within B.code are jumped based on the value of B. If B is true, control flows to the first instruction of S₁.code and if B is false, control flows to the instruction immediately following S₁.code

While generating three address code :-

- *] To generate new symbolic label the function `new-label()` is used.
- *] With the expression `E.true` and `E.false` are the labels associated.
- *] `S.code` and `E.code` is for generating three address code.

if

Semantic rule:- $S \rightarrow \text{if } E \text{ then } S_1$

`E.true = new-label()`

`E.false = S.next`

`S1.next = S.next`

`S.code = E.code || gen.code(E.true) || S1.code`

Consider the statement `if a < b then s = a`

The three address code for if-then is

`if (E) goto L1`
`goto L2`

`L1: S1`

`L2:`

`if a < b goto L1`
`goto L2`

`L1: s = a`

`L2:`

if-else :-

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

Semantic Rule:-

$E.\text{true} = \text{new_label}()$

$E.\text{false} = \text{new_label}()$

$S_1.\text{next} = S.\text{next}$

$S_2.\text{next} = S.\text{next}$

$S.\text{code} = E.\text{code} \parallel \text{gen_code}(E.\text{true}) \parallel S_1.\text{code} \parallel \text{gen_code}$

$(\text{'goto', } S.\text{next}) \parallel \text{gen_code}(E.\text{false}) \parallel S_2.\text{code}$

Ex: Consider the statement if $(a < b)$ then $(s = a)$ else $(s = b)$

The three address code for if then else is

if (E) goto L1

goto L2

L1: S1

goto L3

L2: S2

L3:

if $(a < b)$ goto L1

goto L2

L1: $s = a$

goto L3

L2: $s = b$

L3:

While

$S \rightarrow \text{while } (E) \text{ do } S_1$

Semantic Rule:-

$S.\text{begin} = \text{new_label}()$

$E.\text{true} = \text{new_label}()$

$E.\text{false} = S.\text{next}$

$S_1.\text{next} = S.\text{begin}$

$S.code = gen_code(S.begin) \parallel E.code \parallel gen_code(E.true) \parallel$
 $S1.code \parallel gen_code('goto', S.begin)$

eg:- consider the statement while ($a < b$) do ($s = a$)

while E goto L1
goto L2

L1: S1

L2:

while ($a < b$) goto L1
goto L2

L1: $s = a$

L2:

Generate three address code for the following segment of code.

if ($X < 100 \parallel X \geq 200 \ \&\& \ X \neq Y$) $x = 0$;

if $X < 100$ goto ~~L4~~ L2
goto L3

L3: if $X \geq 200$ goto L4
goto L1

L4: if $X \neq Y$ goto L2
goto L1

L2: $x = 0$

L1: . . .

Syntax - directed definition for flow of Control

Statements

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = new.label()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow assign$	$S.code = assign.code$
$S \rightarrow if(B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow if(B) S_1 \text{ else } S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel$ $S_1.code \parallel gen('goto', S.next) \parallel$ $label(B.false) \parallel S_2.code$
$S \rightarrow while(B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code \parallel label$ $(B.true) \parallel S_1.code \parallel gen$ $('goto', begin)$

Switch Statement :- The Switch or case statement is available in a variety of language. Our switch statement syntax is shown in the fig.(a). There is a selector expression E , which is to be evaluated, followed by n constant values $V_1, V_2, \dots, V_n$ that the expression might take, perhaps including a default "value", which always matches the expression if no other value does.

```
Switch( $E$ ) {  
    case  $V_1$  :  $S_1$   
    case  $V_2$  :  $S_2$   
    ....  
    case  $V_{n-1}$  :  $S_{n-1}$   
    default :  $S_n$   
}
```

fig.(a) : switch-statement syntax

Arrays :- As we know array is a collection of contiguous storage of elements. For accessing any elements of an array what we need is its address. For statically declared arrays it is possible to compute the relative address of each element.

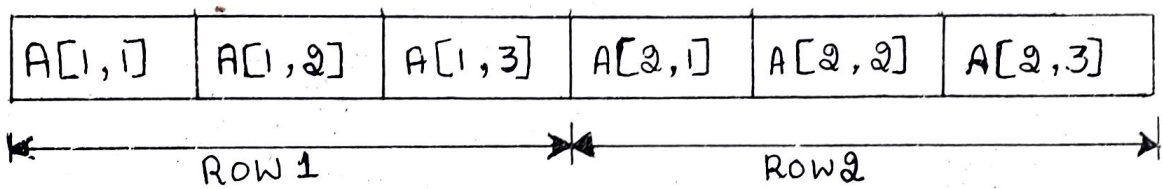
Typically there are two representations of arrays.

1. Row Major Representation.

2. Column Major Representation.

These representations are as shown below

Row Major Representation



Column Major Representation

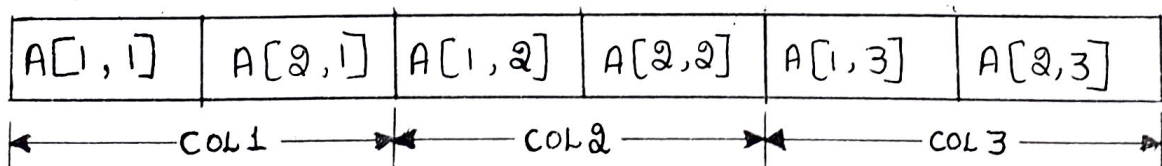


Fig :- Row Major and Column Major Representation

To compute the address of any element:

Let base is the address at a[] and w is the width of the element (required memory units) then to compute ith address of a[]

$$\text{base} + (i - \text{low}) * w$$

where low is lower bound on subscript.

$$\text{base} + (i - \text{low}) * W$$

$$\Rightarrow \text{base} + i * W - \text{low} * W$$

$$\Rightarrow i * W + (\text{base} - \text{low} * W)$$

Let $C = (\text{base} - \text{low} * W)$ is computed at compile time. Then the relative address of $a[i]$ can be computed as

$$C + (i * W)$$

Similarly for calculation of relative address of two dimensional array we need to consider i and j subscripts. Considering row major representation we will compute the relative address for $a[i, j]$ using following formula.

$$a[i, j] = \text{base} + ((i - \text{low}_1) * n_2 + (j - \text{low}_2)) * W$$

where low_1 and low_2 are the two lower bounds on values of i and j .

Assuming that i and j are not known at compile time we can re-write the formula as:

$$a[i, j] = ((i * n_2) + j) * W + (\text{base} - ((\text{low}_1 * n_2) + \text{low}_2) * W)$$

The term $(\text{base} - ((\text{low}_1 * n_2) + \text{low}_2) * W)$ can be computed at compile time.

For example :- Consider an array A of size 10×20 assuming $low_1 = 1$ and $low_2 = 1$. The computation of $A[i, j]$ is possible by assuming that $w = 4$ as

$$A[i, j] = ((i * n_2) + j) * w + (\text{base} - ((low_1 * n_2) + low_2) * w)$$

Given

$$n_2 = 20$$

$$w = 4$$

$$low_1 = 1$$

$$low_2 = 1$$

$$A[i, j] = ((i * 20) + j) * 4 + (\text{base} - ((1 * 20) + 1) * 4)$$

$$A[i, j] = 4 * (20i + j) + (\text{base} - 84)$$

The value $(\text{base} - 84)$ can be computed at compile time.

Question no: 1

Generate the three address code for the expression $x := A[i, j]$ for an array 10×20 . Assume $low_1 = 1$ and $low_2 = 1$

Given that :- $low_1 = 1$, $low_2 = 1$, $n_1 = 10$, $n_2 = 20$

$$A[i, j] = ((i * n_2) + j) * w + (\text{base} - ((low_1 * n_2) + low_2) * w)$$

$$A[i, j] = ((i * 20) + j) * 4 + (\text{base} - ((1 * 20) + 1) * 4)$$

$$\Rightarrow 4 * (20i + j) + (\text{base} - 84)$$

The three address code for this expression can be

$$t_1 = i * 20$$

$$t_1 = t_1 + j$$

$$t_2 = C \quad // \text{Computation of } C = \text{base} - 84 //$$

$$t_3 = 4 * t_1$$

$$t_4 = t_2[t_3]$$

$$X = t_4$$

Question no. 2

Generate the intermediate code for the statement
 $\text{Sum} = A[i, j] + B[i, j]$

Solution :-

Before computing the three address code we will assume $\text{low}_1 = 1$, $\text{low}_2 = 2$, and array is of size 10×10 , $n_1 = 10$ and $n_2 = 10$ and width = 4

$$\begin{aligned} A[i, j] &= ((i * n_2) + j) * W + (\text{base} - ((\text{low}_1 * n_2) + \text{low}_2) * W) \\ &= ((i * n_2) + j) * W + \text{Addr } A \\ &= ((i * 10) + j) * 4 + \text{Addr } A \\ &= (10i + j) * 4 + \text{Addr } A \end{aligned}$$

$$\begin{aligned} B[i, j] &= ((i * n_2) + j) * W + (\text{base} - ((\text{low}_1 * n_2) + \text{low}_2) * W) \\ &= ((i * n_2) + j) * W + \text{Addr } B \\ &= ((i * 10) + j) * 4 + \text{Addr } B \\ &= (10i + j) * 4 + \text{Addr } B \end{aligned}$$

The three address code will then be

$$i = 1$$

$$j = 1$$

$$T_1 = i$$

$$T_2 = j$$

$$T_3 = 10 * T_1$$

$$T_4 = T_3 + T_2$$

$$T_5 = \text{Addr } A$$

$$T_6 = 4 * T_4$$

$$T_7 = T_5[T_6] \quad /* A[i, j] */$$

$$T_8 = 10 * T_1$$

$$T_9 = T_8 + T_2$$

$$T_{10} = \text{Addr } B$$

$$T_{11} = 4 * T_9$$

$$T_{12} = T_{10}[T_{11}] \quad /* B[i, j] */$$

$$T_{13} = T_7 + T_{12} \quad /* A[i, j] + B[i, j] */$$

$$\text{SUM} = T_{13}$$

Back Patching :-

Implementation of syntax directed definition using two pass is the most convenient method. If we decide to generate the three address code for given syntax directed definition using single pass only, then the main problem that occurs is the decision of address of the labels. The goto statements refer there label statements and in one pass it becomes

difficult to know the locations of these label statements. If we use two passes instead of one pass then in one pass we can leave these address unspecified and in the second pass this incomplete information can be filled up. To overcome the problem of processing the incomplete information in one pass the back patching technique is used.

Back Patching Definition :-

Back patching is the activity of filling up unspecified information of labels using appropriate semantic actions in during the code generation process.

To generate code using backpatching. In the semantic actions following functions are used.

1. mklist(i) :- Creates the newlist. The index i is passed as an argument to this function, where i is an index to the array of quadruple.
2. merge_list(p_1, p_2) :- Function concatenates two lists pointed by p_1 and p_2 . It returns the pointer to the concatenated list.
3. backpatch(p, i) :- Insert i as target label for the statement pointed by pointer p .

Backpatching for Boolean Expressions

Consider the grammar for Boolean Expressions

$$B \rightarrow B_1 \parallel M B_2$$

$$B \rightarrow B_1 \&\& M B_2$$

$$B \rightarrow !B_1$$

$$B \rightarrow (B_1)$$

$$B \rightarrow E_1 \text{ relop } E_2$$

$$B \rightarrow \text{True}$$

$$B \rightarrow \text{False}$$

$$M \rightarrow \epsilon$$

Here a new non-terminal M is inserted as a marker non-terminal. The purpose of M is to mark the exact point where the semantic action is picked up.

Production	Semantic Rules
1) $B \rightarrow B_1 \parallel M B_2$	$\{ \text{backpatch}(B_1.\text{falselist}, M.\text{instr});$ $B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist})$ $B.\text{falselist} = B_2.\text{falselist}; \}$
2) $B \rightarrow B_1 \&\& M B_2$	$\{ \text{backpatch}(B_1.\text{truelist}, M.\text{instr});$ $B.\text{truelist} = B_2.\text{truelist};$ $B.\text{falselist} = \text{merge}(B_1.\text{falselist},$ $B_2.\text{falselist}); \}$

Production	Semantic Rules
3] $B \rightarrow !B_1$	$\{ B.\text{truelist} = B_1.\text{falselist};$ $B.\text{falselist} = B_1.\text{truelist}; \}$
4] $B \rightarrow (B_1)$	$\{ B.\text{truelist} = B_1.\text{truelist};$ $B.\text{falselist} = B_1.\text{falselist}; \}$
5] $B \rightarrow E_1 \text{ rel } E_2$	$\{ B.\text{truelist} = \text{makelist}(\text{next instr});$ $B.\text{falselist} = \text{makelist}(\text{next instr} + 1);$ $\text{emit}('if' E_1.\text{addr rel op } E_2.\text{addr}$ $\quad 'goto -');$ $\text{emit}('goto -'); \}$
6] $B \rightarrow \text{true}$	$\{ B.\text{truelist} = \text{makelist}(\text{next instr});$ $\text{emit}('goto -'); \}$
7] $B \rightarrow \text{false}$	$\{ B.\text{falselist} = \text{makelist}(\text{next instr});$ $\text{emit}('goto -'); \}$
8] $M \rightarrow \epsilon$	$\{ M.\text{instr} = \text{next instr}; \}$

Fig:- Translation scheme for boolean Expression

Example :- Using Backpatching, generate an intermediate code for following expression:

$A < B$ OR $C < D$ AND $P < Q$

Solution :-

For the given expression

$A < B$ OR $C < D$ AND $P < Q$ we will scan it from left to right.

$A < B$ matches with the rule $E \rightarrow id, \text{relop } id_2$.

In response to this statement the three address code, will be generated as,

or semantic actions are

100	if $A < B$	goto ____	} append('if' id ₁ . place relop id ₂ . place goto ____) append('goto ____')
101	goto ____		

Similarly for the remaining part of expression the three address code will be

102 if $C < D$ goto ____

103 goto ____

104 if $P < Q$ goto ____

105 goto ____

Now, for the expression

if $A < B$ OR $C < D$ AND $P < Q$, we will solve AND operation first then OR, as AND has higher precedence over OR

Hence,

102 if $C < D$ goto 104

103 goto 107

104 if $P < Q$ goto 106

105 goto 107

Now consider

$A < B$ OR $C < D$ AND $P < Q$

Finally the 3-address with backpatching will be

100 if $A < B$ goto 106

101 goto 102

102 if $C < D$ goto 104

103 goto 107

104 if $P < Q$ goto 106

105 goto 107

106

107

Syntax - Directed Translation of Switch Statement

Code to evaluate E into t
goto test

L_1 : code for S_1
goto next

L_2 : code for S_2
goto next

.....

L_{n-1} : code for S_{n-1}
goto next

L_n : code for S_n
goto next

test : if $t = V_1$ goto L_1
if $t = V_2$ goto L_2
.....
if $t = V_{n-1}$ goto L_{n-1}
goto L_n

next :

Fig.:- Translation of Switch Statement